Transformers

Course of Natural Language Processing

C. M. Bishop, H. Bishop, Deep Learning Foundations and Concepts, Ch. 12

Alberto Castellini

alberto.castellini@univr.it

MS Artificial Intelligence
University of Verona

Slides realized by Alberto Castellini

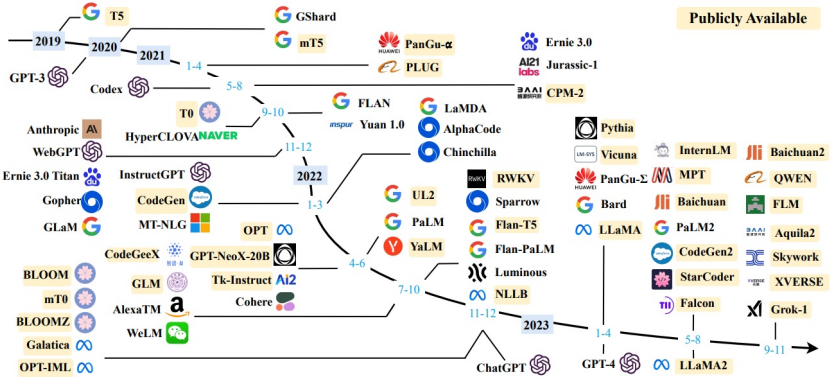
Introduction

- **Transformers** are one of the most important developments in deep learning
- First introduced in **Natural Language Processing (NLP)** they greatly surpassed RNNs
- They are based on a processing concept called **attention**
- They are called **transformers** because they transform input vectors from a representation space to another with **reacher internal representation** for a downstream task
- **Inputs:** unstructured sets of vectors, ordered sequeces, general representations (broad applicability)

Introduction

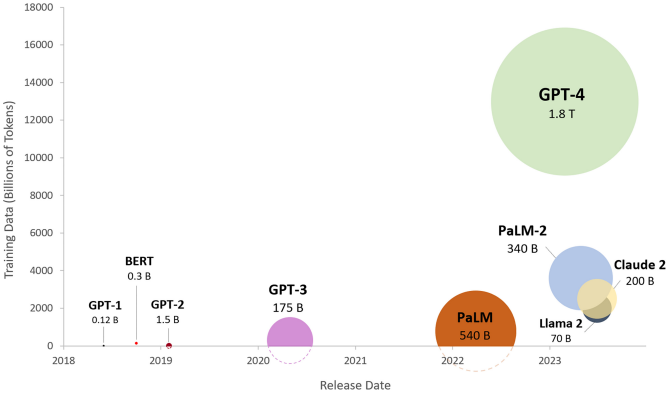
- **Transfer learning** is effective with transformers
- Model trained on a large body of data and applied to several downstream tasks using **fine-tuning** (**foundation model**)
- Transformers can be trained in a **self-supervised** way using unlabelled data
- **Scaling hypothesis**: by increasing the scale of the model (learnable parameters) and the training dataset significant improvements can be achieved
- Transformers are well suited to **massively parallel processing** - very large models (of the order of trillion - 10^{12} - parameters) can be trained in reasonable time

LLM: timeline



Courtesy of Devopedia. https://devopedia.org/large-language-model

LLM: year/size



Courtesy of Devopedia. https://devopedia.org/large-language-model

Presentation Outline

1 Attention [Chapter 12.1, 12.2 Bishop et al.]

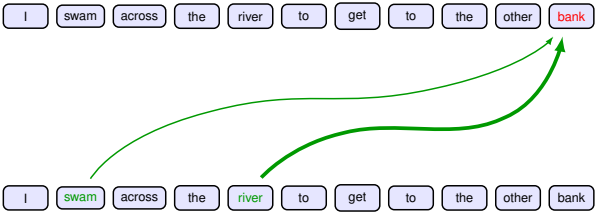
2 Transformer Architectures [Chapter 12.3 Bishop et al.]

Attention

Example. Consider the following two sentences:

- I swam across the river to get to the other bank
- I walked across the road to get cash from the bank

To determine the exact interpretation of the word *bank* an ANN should **rely more heavily on (i.e., attend to)** specific words from the rest of the sentence (**context**)

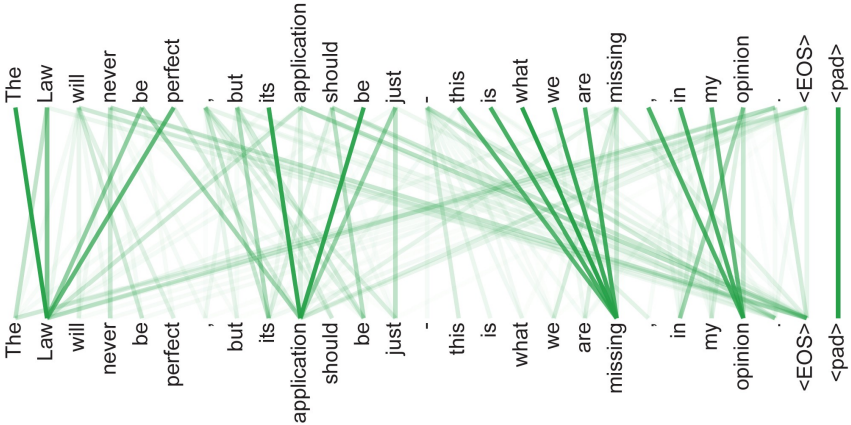


Courtesy of Bishop et al., 2024

Attention

- In standard neural networks, when the network is trained weights are fixed
- By contrast, **attention** uses waiting factors whose values **depend on the specific input data**
- A transformer can be seen as a **rich form of embedding** in which a given vector (e.g., representing a word) is mapped to a location that **depends on the other vectors** (i.e., words) of the sentence
- E.g.: 'bank' close to 'water' in the first sentence, close to 'money' in the second sentence

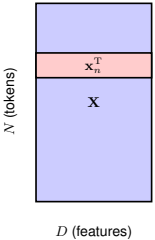
LLM: timeline



Courtesy of Bishop et al., 2024

Transformer processing: notation

- **Input:** set of vectors $\{\boldsymbol{x}_n\}$ of dimensionality D ($n = 1, \ldots, N$)
- Each vector is called **token** (e.g., a word in a sentence, a patch in an image, an aminoacid in a protein, **a number/set of numbers in a time series**, etc.)
- The elements x_{ni} of the token are called **features** ($i = 1, \ldots, D$)
- **Data matrix (set of tokens)** $\boldsymbol{X}$: dimension $N \times D$. Each row is a token vector $\boldsymbol{x}_n^T$. A **training set** contains many sets of tokens (i.e., many matrices)



Courtesy of Bishop et al., 2024

Transformer processing: main building block

- **Transformer layer:** fundamental building block of a transformer that takes a **data matrix** as input and returns a **transformed matrix** of the same dimension as output:

$$\tilde{\boldsymbol{X}} = \mathit{TransformerLayer}[\boldsymbol{X}]$$

- **Multiple transformer layers** can be applied in succession to learn **rich internal representations**
- Each transformer layer contains weights that can be learned by **gradient descent** using an appropriate **cost function**
- Each transformer layer performs two stages:
  1. Mixes features from different tokens across the **columns** of $\boldsymbol{X}$ (attention)
  2. Transforms the values of each token acting on the rows of $\boldsymbol{X}$ independently

Attention coefficients

- Given a set of tokens in an embedding space, $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$
- **Goal:** to map them into another set $\boldsymbol{y}_1, \ldots, \boldsymbol{y}_N$ with richer semantic structure
- The **value** of a generic output token $\boldsymbol{y}_n$ should **depend** on **all** input tokens
- **Attention:** the dependence should be **stronger** for inputs $\boldsymbol{x}_m$ particularly important for determining the new representation of $\boldsymbol{y}_n$
- **Linear combination case:**

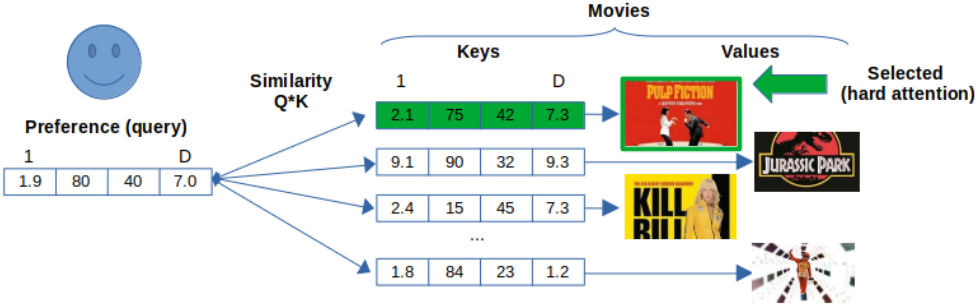
$$\boldsymbol{y}_n = \sum_{m=1}^{N} a_{nm} \boldsymbol{x}_m$$

where a_{nm} are called *attention weights*, with $a_{nm} \geq 0$ and $\sum_{m=1}^{N} a_{nm} = 1$ (partition of unity to avoid compensation).

Self-Attention

How to compute coefficients a_{nm}?

Example (hard attention): movie selection



Self-Attention

In the movie selection example, only the movie (value) with properties (key) more similar to the preferences (query) is selected (**hard attention**)

In transformers:

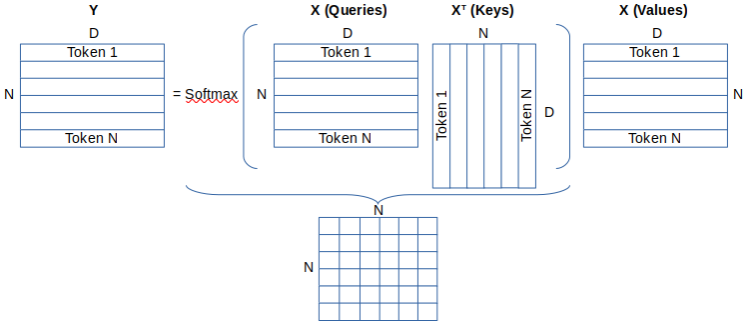
- Each input token $\boldsymbol{x}_n$ is both a **key** and a **value** (i.e., as using the movie itself to summarize the features of the movie)
- The token $\boldsymbol{x}_m$ is used as a **query** for the output $\boldsymbol{y}_m$ (then, $\boldsymbol{x}_m$ is compared to all the key tokens)
- The dot product $\boldsymbol{x}_n^T \boldsymbol{x}_m$ is used to compute the query-key **similarity** which says how much the token represented by $\boldsymbol{x}_n$ should **attend** to the token represented by $\boldsymbol{x}_m$ (i.e., how $\boldsymbol{x}_n$ is important to generate $\boldsymbol{y}_m$)
- Constraints are imposed using the softmax function:

$$a_{nm} = \frac{exp(\boldsymbol{x}_n^T \boldsymbol{x}_m)}{\sum_{m'=1}^{N} exp(\boldsymbol{x}_n^T \boldsymbol{x}_{m'})}$$

Self-Attention

In summary, each input vector $\boldsymbol{x}_n$ is transformed to a corresponding output vector $\boldsymbol{y}_n$ by taking a linear combination of input vectors $\boldsymbol{y}_n = \sum_{m=1}^{N} a_{nm}\boldsymbol{x}_m$ in which the weight a_{nm} applied to vector $\boldsymbol{x}_m$ is computed by the softmax of dot-product between the query $\boldsymbol{x}_n$ and the key $\boldsymbol{x}_m$. In matrix notation:

$$\boldsymbol{Y} = Softmax[\boldsymbol{X}\boldsymbol{X}^T]X$$



Network parameters

Problem:

- The transformation has **no learnable parameters**
- Each of the feature values within a token vector $\boldsymbol{x}_n$ plays an **equal role** in determining the attention coefficients

Solution: define **modified feature vectors** as linear transformations of the original vectors in the form:

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}^{(q)}$$

$$\boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}^{(k)}$$

$$\boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}^{(v)}$$

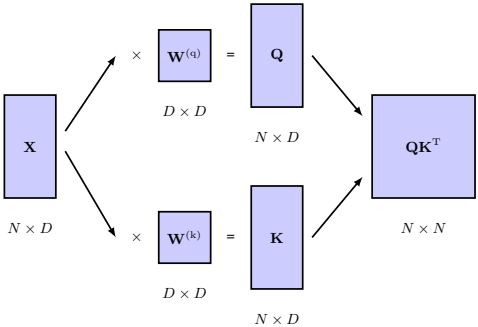
where $\boldsymbol{W}^{(q)}$, $\boldsymbol{W}^{(k)}$, and $\boldsymbol{W}^{(v)}$ are learnable parameters with dimension, respectively, $D \times D_k$ (not D_q), $D \times D_k$, and $D \times D_v$. Usually, $D_k = D_v = D$ so dim output = dim input, and **multiple transformer layers** can be stacked

Network parameters

The general formula for computing the tranformation is

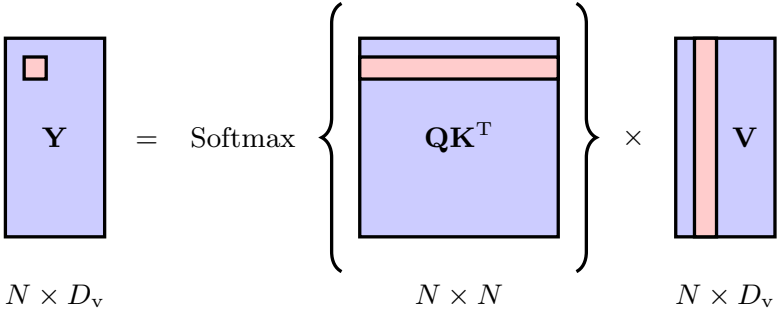
$$\boldsymbol{Y} = Softmax[\boldsymbol{Q}\boldsymbol{K}^T]\boldsymbol{V}$$

where $\boldsymbol{Q}\boldsymbol{K}^T$ has dimension $N \times N$, and the matrix $\boldsymbol{Y}$ has dimension $N \times D_v$



Courtesy of Bishop et al., 2024

Network parameters



Courtesy of Bishop et al., 2024

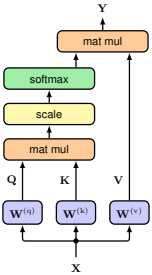
Network parameters

- **Notice:** whereas standard networks multiply activations by **fixed weights**, in transformers the activations are multiplied by **data-dependent** attention coefficients
- **Example:** an attention coefficient could be **close to zero** for an input vector $\boldsymbol{X}'$ (i.e., incoming signal ignored in the signal path) and **very large** for another imput vector $\boldsymbol{X}''$ (i.e., incoming signal considered). By contrast, if a standard neural network learns to ignore an input, it does **for all input vectors**

Scaled self-attention

- **Problem:** gradients of the softmax function become exponentially small for high-magnitude inputs
- **Solution:** rescale $\boldsymbol{QK}^T$ before applying the softmax (scaled dot-product self-attention)

$$\boldsymbol{Y} = Attention(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = Softmax\left[\frac{\boldsymbol{QK}^T}{\sqrt{D_k}}\right]\boldsymbol{V}$$



Scaled self-attention

Algorithm 12.1: Scaled dot-product self-attention

Input: Set of tokens $\mathbf{X} \in \mathbb{R}^{N \times D} : \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$

Weight matrices $\{\mathbf{W}^{(\mathrm{q})}, \mathbf{W}^{(\mathrm{k})}\} \in \mathbb{R}^{D \times D_\mathrm{k}}$ and $\mathbf{W}^{(\mathrm{v})} \in \mathbb{R}^{D \times D_\mathrm{v}}$

Output: $\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \in \mathbb{R}^{N \times D_\mathrm{v}} : \{\mathbf{y}_1, \ldots, \mathbf{y}_N\}$

$\mathbf{Q} = \mathbf{X}\mathbf{W}^{(\mathrm{q})}$ `// compute queries` $\mathbf{Q} \in \mathbb{R}^{N \times D_\mathrm{k}}$

$\mathbf{K} = \mathbf{X}\mathbf{W}^{(\mathrm{k})}$ `// compute keys` $\mathbf{K} \in \mathbb{R}^{N \times D_\mathrm{k}}$

$\mathbf{V} = \mathbf{X}\mathbf{W}^{(\mathrm{v})}$ `// compute values` $\mathbf{V} \in \mathbb{R}^{N \times D}$

return $\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{Softmax}\left[\frac{\mathbf{Q}\mathbf{K}^\mathrm{T}}{\sqrt{D_\mathrm{k}}}\right]\mathbf{V}$

Multi-head attention

- **Problem:** Multiple patterns of attention relevant at the same time
- **Example in NLP:** patterns relevant to **tense** and patterns associated to **vocabulary**
- **Solution:** Use multiple independent attention heads in parallel. With H heads $h = 1, \ldots, H$ of the form

$$\boldsymbol{H}_h = Attention(\boldsymbol{Q}_h, \boldsymbol{K}_h, \boldsymbol{V}_h)$$

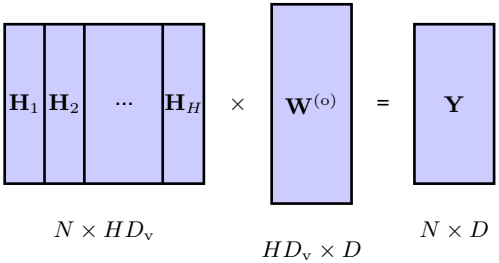
we have separate query, key and value matrices for each head:

$$\boldsymbol{Q}_h = \boldsymbol{X}\boldsymbol{W}_h^{(q)}$$
$$\boldsymbol{K}_h = \boldsymbol{X}\boldsymbol{W}_h^{(k)}$$
$$\boldsymbol{V}_h = \boldsymbol{X}\boldsymbol{W}_h^{(v)}$$

Multi-head attention

The H outputs are first **concatenated** in a single $(N \times HD_v)$ matrix and then **linearly tranformed** using a $(HD_v \times D)$ matrix $W^{(o)}$:

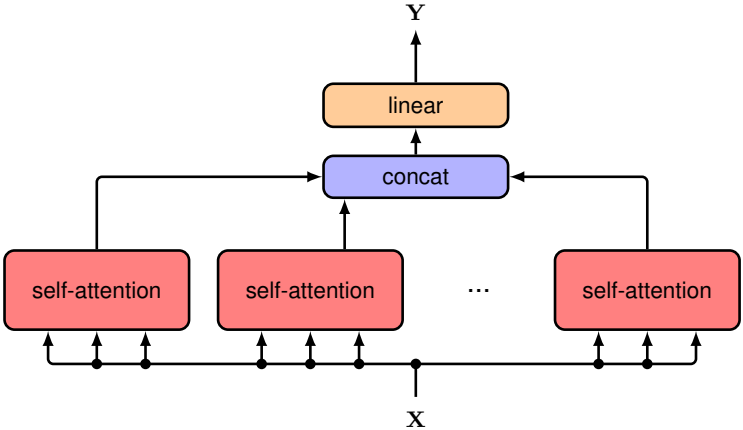
$$\boldsymbol{Y}(\boldsymbol{X}) = Concat[\boldsymbol{H}_1, \ldots, \boldsymbol{H}_H)]\boldsymbol{W}^{(o)}$$



Courtesy of Bishop et al., 2024

Tipically $D_v = D/H$ so the concatenation matrix has dimension $N \times D$

Multi-head attention



Courtesy of Bishop et al., 2024

Multi-head attention

Algorithm 12.2: Multi-head attention

Input: Set of tokens $\mathbf{X} \in \mathbb{R}^{N \times D} : \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$

Query weight matrices $\{\mathbf{W}_1^{(\mathrm{q})}, \ldots, \mathbf{W}_H^{(\mathrm{q})}\} \in \mathbb{R}^{D \times D}$

Key weight matrices $\{\mathbf{W}_1^{(\mathrm{k})}, \ldots, \mathbf{W}_H^{(\mathrm{k})}\} \in \mathbb{R}^{D \times D}$

Value weight matrices $\{\mathbf{W}_1^{(\mathrm{v})}, \ldots, \mathbf{W}_H^{(\mathrm{v})}\} \in \mathbb{R}^{D \times D_{\mathrm{v}}}$

Output weight matrix $\mathbf{W}^{(\mathrm{o})} \in \mathbb{R}^{H D_{\mathrm{v}} \times D}$

Output: $\mathbf{Y} \in \mathbb{R}^{N \times D} : \{\mathbf{y}_1, \ldots, \mathbf{x}_N\}$

`// compute self-attention for each head (Algorithm 12.1)`

for $h = 1, \ldots, H$ **do**

$\mathbf{Q}_h = \mathbf{X}\mathbf{W}_h^{(\mathrm{q})}, \quad \mathbf{K}_h = \mathbf{X}\mathbf{W}_h^{(\mathrm{k})}, \quad \mathbf{V}_h = \mathbf{X}\mathbf{W}_h^{(\mathrm{v})}$

$\mathbf{H}_h = \mathrm{Attention}\left(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h\right)$ `//` $\mathbf{H}_h \in \mathbb{R}^{N \times D_{\mathrm{v}}}$

end for

$\mathbf{H} = \mathrm{Concat}\left[\mathbf{H}_1, \ldots, \mathbf{H}_N\right]$ `// concatenate heads`

return $\mathbf{Y}(\mathbf{X}) = \mathbf{H}\mathbf{W}^{(\mathrm{o})}$

Transformer layers

- Multi-head self-attention is the core of transformer layers
- To improve training efficiency we introduce **residual connections** that bypass the multi-head structure
- Then, **layer normalization** (Ba, Kiros, and Hinton, 2016) is performed, which also improves training efficiency
- Resulting transformation:

$$\boldsymbol{Z} = LayerNorm[\boldsymbol{Y}(\boldsymbol{X}) + \boldsymbol{X}]$$

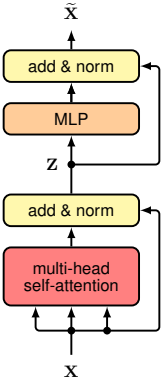
Transformer layers

- **Problem:** the attention mechanism creates **linear combinations** of the value vectors, which are **linearly combined** to produce output vectors. So, **outputs of attention layers are constrained to be linear combinations of the inputs**.
- Nonlinearity enters only in the attention weights (softmax function) but the output vectors are constrained to lie in the subspace spanned by the input vectors, which limits the expressivenes of the attention layer
- **Solution:** We can enhance the flexibility by **post-processing the output of each layer using a standard nonlinear neural network with D inputs and D outputs ($MLP[\cdot]$)**. E.g., two-layer fully connected with ReLU hidden units
- Notice: the same shared network is applied to each of the output vectors (i.e., tokens, rows of $\boldsymbol{Z}$)
- **Residual connections** and **layer normalization** are applied also to this stage, to improve training efficiency

Transformer layers

Final output of the transformer layer:

$$\tilde{\boldsymbol{X}} = LayerNorm[MLP[\boldsymbol{Z}] + \boldsymbol{Z}]$$



Courtesy of Bishop et al., 2024

Transformer layers

Algorithm 12.3: Transformer layer

Input: Set of tokens $\mathbf{X} \in \mathbb{R}^{N \times D} : \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$

Multi-head self-attention layer parameters

Feed-forward network parameters

Output: $\widetilde{\mathbf{X}} \in \mathbb{R}^{N \times D} : \{\widetilde{\mathbf{x}}_1, \ldots, \widetilde{\mathbf{x}}_N\}$

$\mathbf{Z} = \mathrm{LayerNorm}\left[\mathbf{Y}(\mathbf{X}) + \mathbf{X}\right]$ `// Y(X) from Algorithm 12.2`

$\widetilde{\mathbf{X}} = \mathrm{LayerNorm}\left[\mathrm{MLP}\left[\mathbf{Z}\right] + \mathbf{Z}\right]$ `// shared neural network`

return $\widetilde{\mathbf{X}}$

Courtesy of Bishop et al., 2024

Computational complexity of the attention layer (1)

- **Notation**: N vectors (tokens), D dimension
- **Attention layer**:
  - Matrices $\boldsymbol{W}^{(q)}$, $\boldsymbol{W}^{(k)}$, and $\boldsymbol{W}^{(v)}$ have $O(D^2)$ parameters and are shared across input tokens
  - Matrices $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ have $O(ND)$ parameters
  - Total computational cost for attention ($Softmax\left[\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{D_k}}\right]\boldsymbol{V}$): $O(N^2D)$
- **MLP (after attention)**:
  - D inputs, D outputs
  - Computational cost of a forward pass: $O(D^2)$ (for 1 token)
  - Total computational cost (for N tokens): $O(ND^2)$
- Depending on N and D the transformer layer or the MLP layer may dominate the computational cost

Computational complexity: comparison with fully connected network (2)

- **Notation**: N vectors (tokens), D dimension
- **Standard fully connected network**:
  - Input/output size ND
  - Number of independent parameters: $O(N^2D^2)$
  - Computational cost of a forward pass: $O(N^2D^2)$

- Compared to a standard fully connected network (complexity $O(N^2D^2)$) a transformer layer (complexity $\max(O(ND^2), O(N^2D))$ is computationally more efficient

Positional encoding

- The transformer projection is **equivalent with respect to input permutations**.
- Namely, permuting the order of the rows of $\boldsymbol{X}$ we get an equivalent permutation in the rows of $\widetilde{\boldsymbol{X}}$
- This is a limitation in **NLP** where **token order is crucial**. E.g.,
  - *The food was bad, not good at all*
  - *The food was good, not bad at all*
- A way is needed to **inject token order** information into the network

Positional encoding

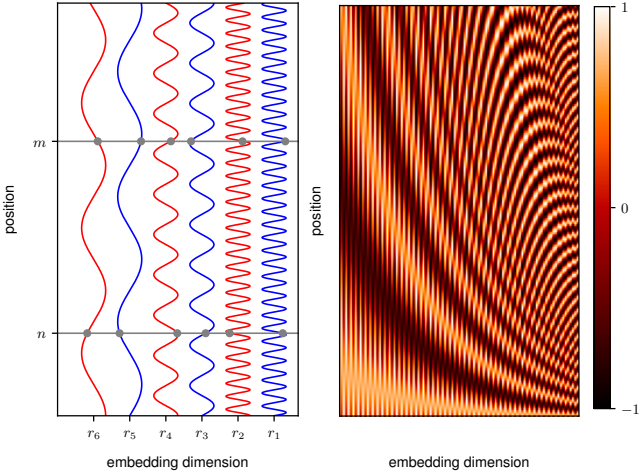
Idea:

- Construct a **position encoding vector** $\boldsymbol{r}_n$ associated with each input position $n \in 1, \ldots, N$
- Add the position vector to the token vector:

$$\widetilde{\boldsymbol{x}}_n = \boldsymbol{x}_n + \boldsymbol{r}_n$$

- **Why doesn't it corrupt the tokens?**
  - Intuition: in the **highly dimensional** space of token embedding it is highly probable that the token and the positional encoding are **uncorrelated**
  - The network is able to process the two information separately
- **Ideal position embedding:** *i)* unique for each position, *ii)* bounded, *iii)* generalize to longer sequences, *iv)* consistent in relative positions.

Positional encoding: sinusoidal functions



Courtesy of Bishop et al., 2024

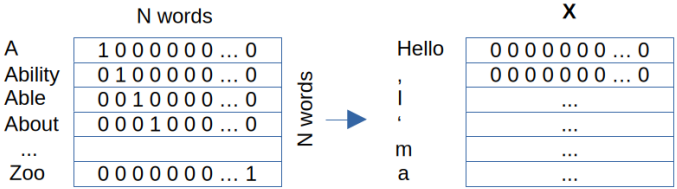
$$r_{ni} = \begin{cases} \sin\left(\dfrac{n}{L^{i/D}}\right), & \text{if } i \text{ is even,} \\ \cos\left(\dfrac{n}{L^{(i-1)/D}}\right), & \text{if } i \text{ is odd.} \end{cases}$$

Tokenization: one-hot representation

How can we compute the token vectors for a given text?
E.g., *"Hello, I'm a language model,"*

One-hot representation

- Fixed dictionary of words
- Vectors of length equal to the dictionary size
- k-th word has 1 in position k, 0 elsewhere
- **Problem:** huge dimensionality of the dictionary and the tokens



Tokenization: Byte Pair Encoding

- One-hot (word) encoding has too large dictionary
- We could use a **dictionary of characters** (instead of words) $\rightarrow$ Disadvantage: it would discard the **semantically important word structure of language**.

Byte Pair Encoding: intermediate solution (between character and word dictionary). Compression method adapted to tokenization.

1. Start with individual **characters**
2. Search a body of text for the **most frequently occurring adjacent pairs** of tokens
3. **Replace** them with a new token and **add** the token to the dictionary (new tokens cannot contain spaces)
4. **Repeat** until a predefined number of tokens is reached

Given a new text, tokenize it by searching for the longest possible tokens first.

Byte Pair Encoding - Dictionary generation

Peter Piper picked a peck of pickled peppers

Peter Piper picked a peck of pickled peppers

Peter Piper picked a peck of pickled peppers

Peter Piper picked a peck of pickled peppers

Peter Piper picked a peck of pickled peppers

Peter Piper picked a peck of pickled peppers

Byte Pair Encoding - Text Tokenization

E.g., *"Hello, I'm a language model,"* → [15496, 11, 314, 479, 257, 3306, 3560, 11]

Token ID	Token (BPE piece)	Meaning / Note
15496	`Hello`	Word without leading space
11	`,`	Comma
314	`ĠI`	Leading-space marker → " I"
479	`'m`	Apostrophe + m
257	`Ġa`	" a"
3306	`Ġlanguage`	" language"
3560	`Ġmodel`	" model"
11	`,`	Final comma

Presentation Outline

1 Attention [Chapter 12.1, 12.2 Bishop et al.]

2 Transformer Architectures [Chapter 12.3 Bishop et al.]

Transformer Architectures

The flexibility of transformer processing layers have allowed the development of **Large Language Models (LLM)**

Three main **transformer-based architectures**:

- **Encoder.** Input: sequence of tokens (e.g., text); output: sequence of tokens (embedding) usable for a downstream task (e.g., sentiment classification).
- **Decoder.** Input: single vector (e.g., text prompt or image); output: sequence of tokens (e.g., next text or image caption).
- **Encoder+Decoder.** Input: sequence (e.g., sequence in italian); output: sequence (e.g., sequence in english)

Decoder transformers: introduction

- **Decoder transformer:** usually used as **generative models** that create **output sequences of token** (e.g., GPT, generative pre-trained transformer (Radford et al., 2019; Brown et al., 2020; OpenAI, 2023)).
- Used to construct an **autoregressive model** of the form

$$p(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N) = \prod_{n=1}^{N} p(\boldsymbol{x}_n | \boldsymbol{x}_1, \ldots, \boldsymbol{x}_{n-1})$$

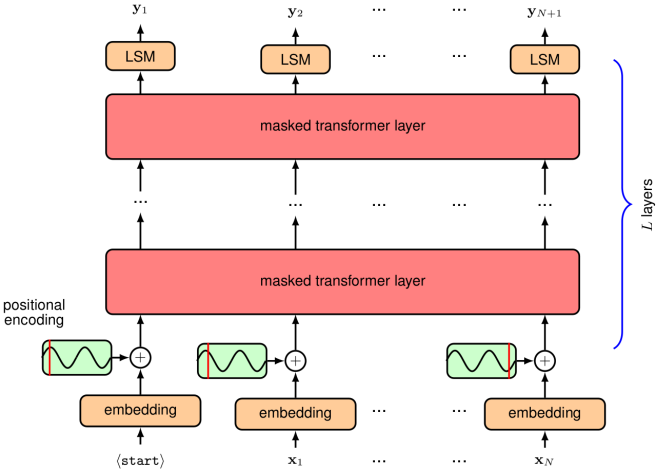
  where the conditional distributions $p(x_n | x_1, \ldots, x_{n-1})$ are expressed using a **transformer neural network** learned from data.
- **Input:** sequence of $n-1$ tokens
- **Output:** conditional distribution for token n
- **Samping** from this distribution we extend the sequence to n tokens. The new sequence can be **fed back** to get a distribution over token $n+1$, and so on

Decoder transformers: architecture

The **architecture** of a decoder transformer consists of:

- a **stack of transformer layers** that take a sequence $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$ of tokens, each of dimensionality D, as **input** and produce a sequence $\tilde{\boldsymbol{x}}_1, \ldots, \tilde{\boldsymbol{x}}_N$ of tokens, again of dimensionality D, as **output**;
- a **linear transformation** (LSM) of each output token followed by a **softmax activation function** to represent a **probability distribution** over the dictionary of tokens at that time step.

Decoder transformers: architecture



Courtesy of Bishop et al., 2024

Decoder transformers: architecture (continue)

- The **linear transformation** (L) of each output token $\tilde{\boldsymbol{x}}_i, i = 1, \ldots, N$ is obtained using a matrix $\boldsymbol{W}^{(p)}$ of dimensionality $D \times K$ (it maps the D dimensions of the token into K values (probabilities));
- The generation of the **probability distribution** is obtained by a **softmax activation function** (SM), namely

$$\boldsymbol{Y} = Softmax(\tilde{\boldsymbol{X}}\boldsymbol{W}^{(p)})$$

where $\boldsymbol{Y}$ is a matrix whose nth row is y_n^T, and $\tilde{\boldsymbol{X}}$ is a matrix whose nth row is $\tilde{\boldsymbol{x}}_n^T$. Each softmax output unit has an associated **cross-entropy error function**.

Decoder transformers: training

- The model can be **trained** using a large corpus of **unlabelled natural language** by **self-supervised learning**
- **Training sample:** a sequence of tokens $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n$ (input to the network) and an associated target value $\boldsymbol{x}_{n+1}$ (next token in the sequence)
- **Error function:** sum of the **cross-entropy error values** summed over the training set and grouped into appropriate **mini-batches**. The sequences are considered independent and identically distributed.
- Each sample can be processed independently from the others but greater efficiency is obtained by processing an entire sequence at once.

Decoder transformers: training (continue)

Consider the word **sequence**

I swam across the river to get to the other bank.

possible **samples** are

- 'I swam across' $\rightarrow$ 'the'
- 'I swam across the' $\rightarrow$ 'river'

Problem: To process them **in parallel**, the network must **not be able to 'cheat'** by looking ahead in the sequence (otherwise it will simply copy the next input).

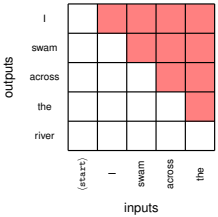
1. We **shift the input sequence to the right** by one step (input x_n corresponds to output y_{n+1}, with target x_{n+1}, and add an additional special token $\langle start \rangle$
2. We introduce **masked attention**

Masked attention

- We set to **zero** all of the **attention weights** that correspond to a token attending to any **later token** in the sequence.
- I.e., setting to zero all the corresponding elements of the attention matrix

$$Attention(Q, K, V)$$

and then normalizing the remaining elements so that each row once again sums to one



Courtesy of Bishop et al., 2024

Exploiting GPU massive parallelism

- To explot massive parallelism of GPUs **multiple sequences** may be **stacked together** into an **input tensor** for **parallel processing in a single batch**
- This requires the sequences to be of the **same length**, whereas text sequences naturally have variable length
- Unused positions are then filled with the $\langle pad \rangle$ token.
- An **additional mask** is then used in the **attention weights** to ensure that the output vectors do **not pay attention** to any inputs occupied by the $\langle pad \rangle$ token.

Usage of the trained model

- The **output of the trained model** is a **probability distribution** over the space of **tokens**, given by the **softmax** output activation function, which represents the **probability of the next token given the current token sequence**.
- Once the **next word is chosen**, the token sequence with the new token included can then be **fed through the model** again to generate the **subsequent token** in the sequence, and this process can be **repeated indefinitely** or until an end-of-sequence token is generated
- **Notice:** due to the masked attention, the **embedding** learned for a particular **token depends only on that token itself and on earlier tokens**, hence it **does not change when a new, later token is generated**. I.e., much of the computation can be recycled when processing a new token.

Sampling strategies

- **Greedy search:** to select the token with the highest probability (deterministic model, i.e., a given input sequence always generates the same output sequence)
- Observation: choosing the **highest probability token** at each stage is **not** the same as selecting the **highest probability sequence of tokens**
- To find the **most probable sequence** we would need to **maximize the joint distribution over all tokens**

$$p(\boldsymbol{y}_1, \ldots, \boldsymbol{y}_N) = \prod_{n=1}^{N} p(\boldsymbol{y}_n | \boldsymbol{y}_1, \ldots, \boldsymbol{y}_{n-1})$$

- The total number of sequences is $O(K^N)$, where K is the dimension of the token vocabulary and N is the sequence length. Greedy search has instead a cost $O(KN)$, linear in the sequence length.

Sampling strategies

Sampling strategies, aim to balance complexity and quality of the selected output sequences. Some examples are:

- Beam search
- Nucleous sampling

Refer to the book (Sec. 12.3.2) for details.

Encoder transformers: architecture

Encoder transformer:

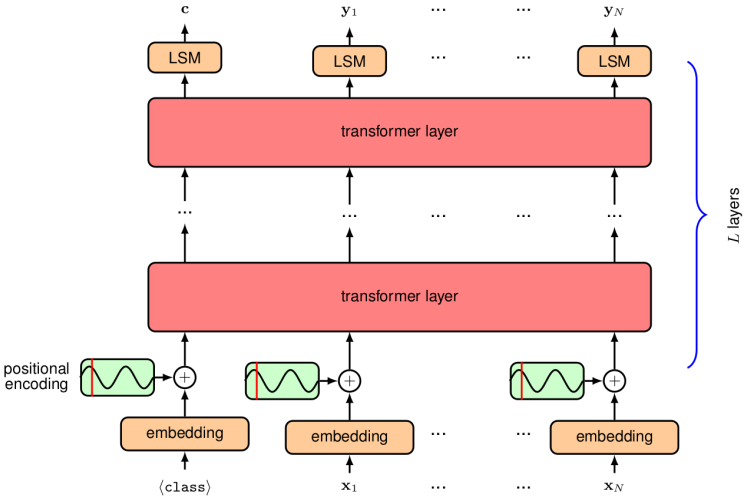
- input: **sequence** of tokens
- output: a sequence of tokens (embedding) usable for a downstream task and a **fixed-length vector** (e.g., a class label)

Example: *Bidirectional Encoder Representations from Transformers (BERT)* (Devlin et al. 2018).

Goal:

- **Pre-train** a language model on a large corpus of text
- **Fine-tune** the model on downstream tasks using **transfer learning** with small **supervised training set**

Encoder transformers: architecture



Courtesy of Bishop et al., 2024

Encoder transformers: training

Pre-training

- **Input:** token sequences (e.g., sentences in LLM). A randomly chosen set of the tokens (e.g., 15%) are replaced by a $\langle mask \rangle$ special token. The class output is ignored during pre-training. Example:

  I $\langle mask \rangle$ across the river to get to the $\langle mask \rangle$ bank.

- **Output:** The model is trained to predict the missing tokens at the corresponding output nodes (**self-supervised learning**). E.g., the network should predict 'swam' at output node 2 and 'other' at output node 10 (only these nodes contribute to the **error function**)
- A large corpus of text is used. The pre-training process takes time and resources

Encoder transformers: comparison with decoder

- The network sees words both **before and after the masked word** (Bidirectional=Not masked) and can use both sources of information to make a prediction, hence there is **no need** to **shift** the inputs to the right by one place
- There is **no need** to **mask** the outputs of each layer from seeing input tokens occurring **later** in the sequence
- An encoder is **less efficient** since only a fraction of the sequence tokens are used as training labels
- An encoder model is **unable to generate sequences**

Encoder transformers: training (continue)

Fine-tuning

- Once the encoder model is pre-trained it can be fine-tuned for a **variety of different downstream tasks**
- A new **output layer** is constructed whose form is **specific** to the task being solved
- Example: **text classification**. Only the first output position is used, which corresponds to the $\langle class \rangle$ token

Encoder transformers: training (continue)

Fine-tuning (continue)

- If the $\boldsymbol{c}$ output has dimension D then it is multiplied by a **matrix** of parameters of dimension $D \times K$, where K is the number of classes. This feeds into a K-dimensional **softmax function** or a vector of dimension $\mathrm{D} \times 1$ followed by a logistic sigmoid for $\mathrm{K} = 2$. The **linear output transformation** could alternatively be replaced with a more complex differentiable model such as an **MLP**
- During fine-tuning **all model parameters** including the **new output matrix** are learned by **stochastic gradient descent** using the log probability of the correct label
- The output of a pre-trained model might feed into a **sophisticated generative deep learning model** for applications such as **text-to-image**

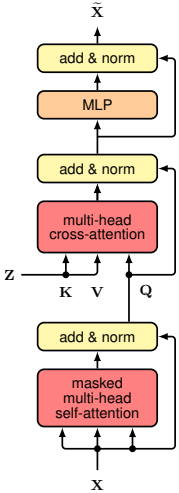
Sequence-to-sequence transformers

- Model defined in Vaswani et al. (2017)
- It combines an encoder with a decoder
- E.g., translating an English sentence into a Dutch sentence
- **Decoder model:** used to generate the token sequence corresponding to the Dutch output, token by token
- Problem: this output needs to be **conditioned** on the **entire input sequence** corresponding to the **English sentence**
- **Encoder transformer:** used to map the **input token sequence** into a suitable **internal representation**, which we denote by $\boldsymbol{Z}$.
- To **incorporate** $\boldsymbol{Z}$ into the **generative process** for the output sequence, we use a modified form of the attention mechanism called **cross attention**.

Cross-attention

- Model defined in Vaswani et al. (2017)
- It combines an encoder with a decoder
- E.g., translating an English sentence into a Dutch sentence
- **Decoder model:** used to generate the token sequence corresponding to the Dutch output, token by token
- Problem: this output needs to be **conditioned** on the **entire input sequence** corresponding to the **English sentence**
- **Encoder transformer:** used to map the **input token sequence** into a suitable **internal representation**, which we denote by $\boldsymbol{Z}$.
- To **incorporate** $\boldsymbol{Z}$ into the **generative process** for the output sequence, we use a modified form of the attention mechanism called **cross attention**.

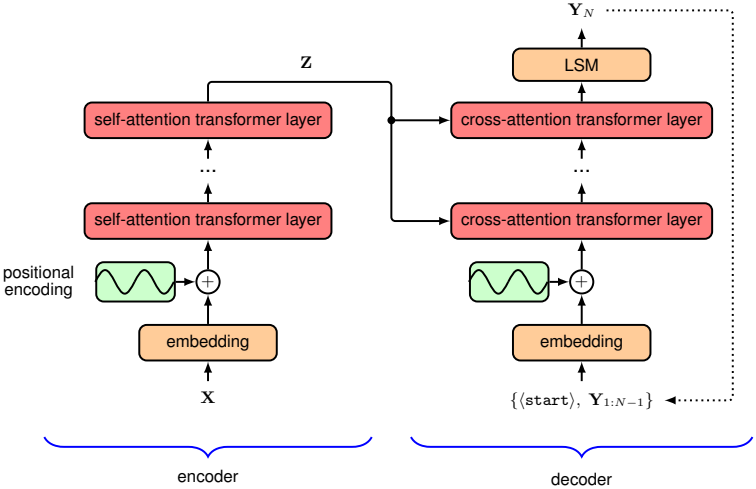
Cross-attention transformer layer



Courtesy of Bishop et al., 2024

- The **query** vectors come from the **sequence being generated**, e.g., the Dutch output sequence
- The **key** and **value** vectors come from the **sequence represented by** $\boldsymbol{Z}$

Sequence-to-sequence transformer



Courtesy of Bishop et al., 2024

Large language models

- LLM: very large **transformer-based neural networks** for **natural language processing**. The most important recent development in the field of machine learning.
- **Number of weight and bias parameters** in the network, up to around some trillion (10^{12})
- **Expansive to train:** they need massively parallel training hardware based on GPUs and large datasets
- The number of compute operations required to train a state-of-the-art machine learning model has grown exponentially since about 2012 with a doubling time of around 3.4 months
- Large language models are trained by **self-supervised learning** on very large data sets of text with potentially other token sequences such as computer code

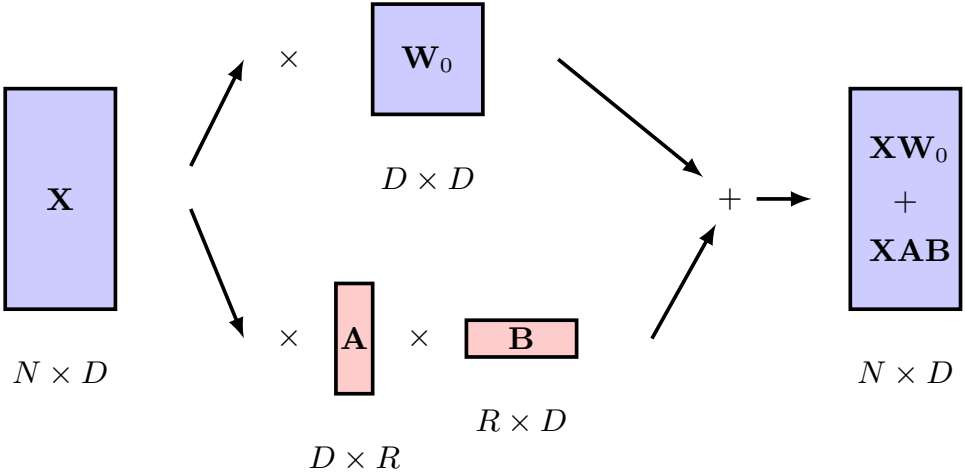
Large language models (continue)

- **Paradigm shift:** large models are first **pre-trained** using **unlabelled data** and then subsequently **fine-tuned** using **supervised learning** based on a much **smaller set of labelled data**
- **Foundation models** (Bommasani et al., 2021): model with broad capabilities that can be subsequently fine-tuned for specific tasks ('downstream' applications).
- **Fine-tuning** can be done by **adding extra layers to the outputs** of the network or by **replacing the last few layers** with fresh parameters and then using the **labelled data to train these final layers**
- Fine-tuning: the weights and biases in the **main model** can either be left **unchanged** or be allowed to undergo **small levels of adaptation**. Typically the **cost** of the fine-tuning is **small** compared to that of pre-training.

Low-rank adaptation

- **LoRA** (Hu et al., 2021): very efficient approach to fine-tuning
- **Idea:** a trained over-parameterized model has a low intrinsic dimensionality with respect to fine-tuning
- LoRA **freezes** the weights of the **original model** and adds **additional learnable weight matrices** into each layer of the transformer in the form of **low-rank products**
- Typically **only attention-layer weights are modified**, whereas MLP-layer weights are kept fixed.

Low-rank adaptation



Courtesy of Bishop et al., 2024

Low-rank adaptation

- The number of parameters in the additional weight matrix $\boldsymbol{AB}$ is $2RD$ compared to the D^2 parameters in the original weight matrix $\boldsymbol{W}_0$, and so if $R \ll D$ then the **number of parameters that need to be adapted during fine-tuning is much smaller** than the number in the original transformer.
- In practice, this can **reduce the number of parameters** that need to be **trained** by a **factor of 10,000**.
- Once the fine-tuning is complete, the **additional weights can be added to the original weight matrices** to give a new weight matrix:

$$\hat{\boldsymbol{W}} = \boldsymbol{W}_0 = \boldsymbol{AB}$$

- During **inference** there is **no additional computational overhead**

Prompt engineering

- As language models have become larger and more powerful, the **need for fine-tuning has diminished**
- If a text string

  English: the cat sat on the mat. French:

  is given as the input sequence (**prompt**), an autoregressive language model can **continue** to generated tokens represent the **French translation**
- A user can **interact with such models using a natural language dialogue**
- **Fine-tuning** large language models through **human evaluation** of generated output, using methods such as **reinforcement learning through human feedback** or **RLHF** (Christiano et al., 2017), has created large language models with impressively easy-to-use conversational interfaces (e.g., **OpenAI ChatGPT**).

Prompt engineering

- **Promt:** sequence of input tokens given by the user (e.g., opening words of a story, which the model is required to complete; question, and the model should provide the answer)
- By using **different prompts**, the same trained neural network may be capable of solving a **broad range of tasks** such as generating **computer code** from a simple text request or writing **rhyming poetry** on demand.
- **Prompt engineering** (Liu et al., 2021): design a good form for a prompt that results in high-quality output for the downstream task
- **Prefix prompt:** by pre-pending an additional token sequence to the prompt the user can modify the form of the output (e.g., 'not to include offensive language')
- **Few-shot learning:** allowing the model to **solve new tasks** simply by **providing some examples within the prompt, without needing to adapt the parameters of the model**

Some Further Readings

Deep Learning, Goodfellow, Bengio, Courville.
Efficient BackProp, LeCun et al.
Deep Residual Learning for Image Recognition, He et al.